

DATA SCIENCE



ON-CAMPUS PLACEMENT PREDICTION

SUMMER INTERNSHIP PROJECT

Submitted by

ABDUL RAHEEM K

(730322243001)

FIFTH SEMESTER

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

KANGEYAM INSTITUTE OF TECHNOLOGY,

KANGEYAM

JUNE 2026



KANGEYAM
Institute of Technology

Approved by AICTE, New Delhi | Affiliated to Anna University, Chennai
ISO Certified Institution | Recognised u/s 2(f) of UGC
Accredited by NBA : UG : CIVIL, CSE, ECE, EEE & MECH
EBET Knowledge Park, Nathakadaiyur, Kangeyam,
Tiruppur-638 108, Tamil Nadu, India.

AUTONOMOUS



ACCREDITED INSTITUTION
with 3.36 CGPA on 4.0 scale



NATIONAL BOARD
of ACCREDITATION

www.kitech.edu.in

BONAFIDE CERTIFICATE

Certified that this is a Bonafide record of work done by **ABDUL RAHEEM K (23AID001)** on AD3512- Summer Internship during the Academic year 2025-26 for fifth Semester.

STAFF –INCHARGE

Ms.M. Raaji BE.,ME.,
Assistant Professor,
Department of Artificial Intelligence and
Data Science,
Kangeyam Institute Of Technolgy

HEAD OF THE DEPARTMENT

Dr.S.Selvi BE.,ME.,P.hd
Associate Professor,
Department of Artificial Intelligence
and Data Science,
Kangeyam Institute Of Technolgy

Submitted for the internship project in Data Science viva voce held on.....

INTERNALEXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to express our deep and heartfelt thanks to our respected Chairman **Thiru. N. RAMALINGAM**, Honorable Secretary **Thiru C. K. VENKATACHALAM**, beloved Correspondent **Thiru S. ANANDAVADIVEL** and our responsible Treasurer **Thiru C. K. BALASUBRAMANIAM** for giving motivation and providing all necessary facilities for successful completion of the Internship project.

We would like to express our gratitude to **Dr. S. RAMKUMAR, M.E., Ph.D.**, Principal, for the moral support and deeds in bringing out this Internship project successfully.

We express our gratitude to **Dr. S. SELVI B.E., M.E., Ph.D.**, Associate Professor & Head of the Department, Department of Artificial Intelligence and Data Science and our Internship Project Coordinator for encouraging a lot throughout this work.

We highly obliged to the Internship Project Supervisor **Ms. M. RAAJI B.E., M.E.**, Assistant Professor, Department of Artificial Intelligence and Data Science for his guidance, suggestions and active encouragement for the fruitful completion of Internship project.

We would also like to thank and express our deep regards to all **Teaching faculty members and non-teaching staff members** of our department. Our sincere thanks to our **Parents** and **Friends** for their continuous encouragement and the untiring support rendered to us in all deeds and walks of this Internship project.

TABLE OF CONTENTS		
CHAPTER	TITLE	PAGE NO
1	INTRODUCTION • 1.1 AIM • 1.2 PURPOSE • 1.3 USE CASES • 1.4 OBJECTIVES	6
2	DESCRIPTION • 2.1 PROJECT DESCRIPTION • 2.2 MODULE DESCRIPTION	8
3	SYSTEM SPECIFICATIONS • 3.1 SOFTWARE REQUIREMENTS	9
4	CONCLUSION AND FUTURE SCOPE • 4.1 CONCLUSION • 4.2 FUTURE SCOPE	10
5	APPENDICES • A.1 SOURCE CODE • A.2 SCREENSHOTS	11
	REFERENCES	23

ABSTRACT

ON CAMPUS PLACEMENT PREDICTION

On-campus placements are a critical milestone in a student's academic journey, directly influencing their career trajectory. This project aims to predict the placement status of students using machine learning techniques, thereby assisting both students and institutions in making informed decisions. Implemented in Python using libraries such as scikit-learn and pandas, the model analyzes student data including academic scores, technical skills, and other relevant factors. The dataset undergoes thorough preprocessing, including handling missing values and encoding categorical variables to ensure compatibility with machine learning algorithms. Multiple models, including Logistic Regression, Decision Trees, and Random Forests, are trained and evaluated using performance metrics such as accuracy, precision, recall, and confusion matrix analysis. The best-performing model is identified for potential deployment. This system provides early insights for students to improve employability and enables institutions to understand placement trends, thereby enhancing training programs and support mechanisms.

CHAPTER 1

INTRODUCTION TO ON-CAMPUS PLACEMENT PREDICTION

1.1 AIM

The aim of this project is to develop a machine learning-based prediction system for on-campus placements using Python. It preprocesses student academic data, applies relevant machine learning algorithms, and predicts placement outcomes to guide students and colleges for better placement preparation and analysis.

1.2 USE CASES

- Predict placement eligibility for students based on their academic records.
- Assist institutions in identifying patterns in placement data.
- Guide training and skill enhancement sessions based on prediction trends.

Enable data-driven insights for placement cell planning.

1.3 PURPOSE

The purpose is to leverage data-driven prediction models to help students understand the likelihood of getting placed based on academic and profile parameters, aiding institutions in improving their training plans while enabling students to identify areas for improvement to increase their placement chances.

1.4 OBJECTIVES

- **Data Collection and Preprocessing:** Collect student academic and placement data, handle missing values, and encode categorical features.
- **Feature Engineering:** Identify relevant features such as CGPA, SSC, HSC percentages, and employability test scores for training models.
- **Model Development:** Train models using Logistic Regression, Decision Trees, and Random Forests for prediction.
- **Evaluation:** Evaluate models using accuracy, precision, recall, and confusion matrix.
- **Deployment Ready:** Prepare the system for real-time placement prediction integration within institutional portals.

CHAPTER 2

DESCRIPTIONS

2.1 PROJECT DESCRIPTION

This project predicts the on-campus placement status of students using Python-based machine learning pipelines. The dataset includes academic scores, employability test results, and categorical features like gender and work experience. Data preprocessing ensures the removal of null values and proper encoding for algorithm compatibility. Feature engineering refines input features to enhance predictive performance. Logistic Regression, Decision Trees, and Random Forests are trained to classify students as ‘Placed’ or ‘Not Placed’. Model performance is evaluated using accuracy and confusion matrices, identifying the best model for deployment. The system is scalable and can be integrated into institutional workflows to improve placement guidance and analytics.

2.2 MODULE DESCRIPTION

- **Data Collection and Import:** Loading CSV data using pandas for processing.
- **Data Preprocessing:** Cleaning null values, encoding categorical data using LabelEncoder, and preparing data for training.
- **Feature Selection:** Selecting critical features such as SSC%, HSC%, CGPA, and employability test scores for prediction.
- **Model Training:** Using Logistic Regression, Decision Trees, and Random Forest classifiers from scikit-learn to build predictive models.
- **Model Evaluation:** Evaluating models with accuracy, precision, recall, and confusion matrix.
- **Prediction and Reporting:** Predicting placement outcomes and providing interpretable reports for analysis.

CHAPTER 3

3.1 SOFTWARE REQUIREMENTS

- **Operating System:**

Windows 10/11 or Linux (64-bit)

- **Python Environment:**

Anaconda/Miniconda or standalone Python 3.11+

- **Libraries:**

pandas, numpy, scikit-learn, matplotlib, seaborn

- **Optional:**

Jupyter Notebook or VS Code for execution

CHAPTER 4

CONCLUSION AND FUTURE SCOPE

5.1 CONCLUSION

This project demonstrates the potential of using machine learning to predict on-campus placements effectively. By analyzing academic and profile features, the system predicts student placement outcomes with significant accuracy, providing valuable insights to students and institutions. This supports early intervention, training adjustments, and informed decision-making to enhance placement outcomes.

5.2 FUTURE SCOPE

- Extend the dataset with additional employability metrics and soft-skill evaluations for improved prediction.
- Develop a web-based interface for students to check their placement prediction status.
- Integrate explainable AI modules to interpret model decisions.
- Incorporate continuous model retraining with new batch data.
- Explore ensemble models for more accuracy.

CHAPTER 5

APPENDICES

A.1 SOURCE CODE Python scripts developed in the Jupyter notebook including preprocessing, model training, evaluation, and prediction pipelines.

#Import Necessary Libraries

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

#Load Dataset

df=pd.read_csv(r"/content/placementdata.csv")

df

#Data Cleaning & Data Preprocessing

df.info()

df.columns

df.drop(columns=['StudentID'], inplace=True)

df.isna().sum()

df.duplicated().sum()

df=df.drop_duplicates()

from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

le.fit(df['ExtracurricularActivities'])

```

df['ExtracurricularActivities'] = le.transform(df['ExtracurricularActivities'])
le.fit(df['PlacementTraining'])
df['PlacementTraining'] = le.transform(df['PlacementTraining'])
le.fit(df['PlacementStatus'])
df['PlacementStatus'] = le.transform(df['PlacementStatus'])
df
#Analysis with Plots
sns.pairplot(df, hue='PlacementStatus', diag_kind='kde')
for i in df.columns:
    if i != 'PlacementStatus':
        plt.figure(figsize=(12, 8))
        plt.title(f'Boxplot of {i} by PlacementStatus')
        sns.boxplot(x='PlacementStatus', y=i, data=df)
        plt.show()
#Split the Feature and Target
x=df.drop(columns=['PlacementStatus'])
y=df['PlacementStatus']
#Scaling the Data
from sklearn.preprocessing import RobustScaler
sc = RobustScaler()
sc.fit(x)
x_scaled = sc.transform(x)
x_scaled

```

#Split the Training and Test Data

from sklearn.model_selection import train_test_split

*x_train, x_test, y_train, y_test = train_test_split(x_scaled, y, test_size=0.2,
random_state=16)*

#Fit the Data into the models

#RandomForestClassifier

*x_train, x_test, y_train, y_test = train_test_split(x_scaled, y, test_size=0.2,
random_state=42)*

from sklearn.ensemble import RandomForestClassifier

*model = RandomForestClassifier(criterion='gini',max_features='sqrt',
random_state=0)*

model.fit(x_train, y_train)

#Predict the Target and Print accuracy

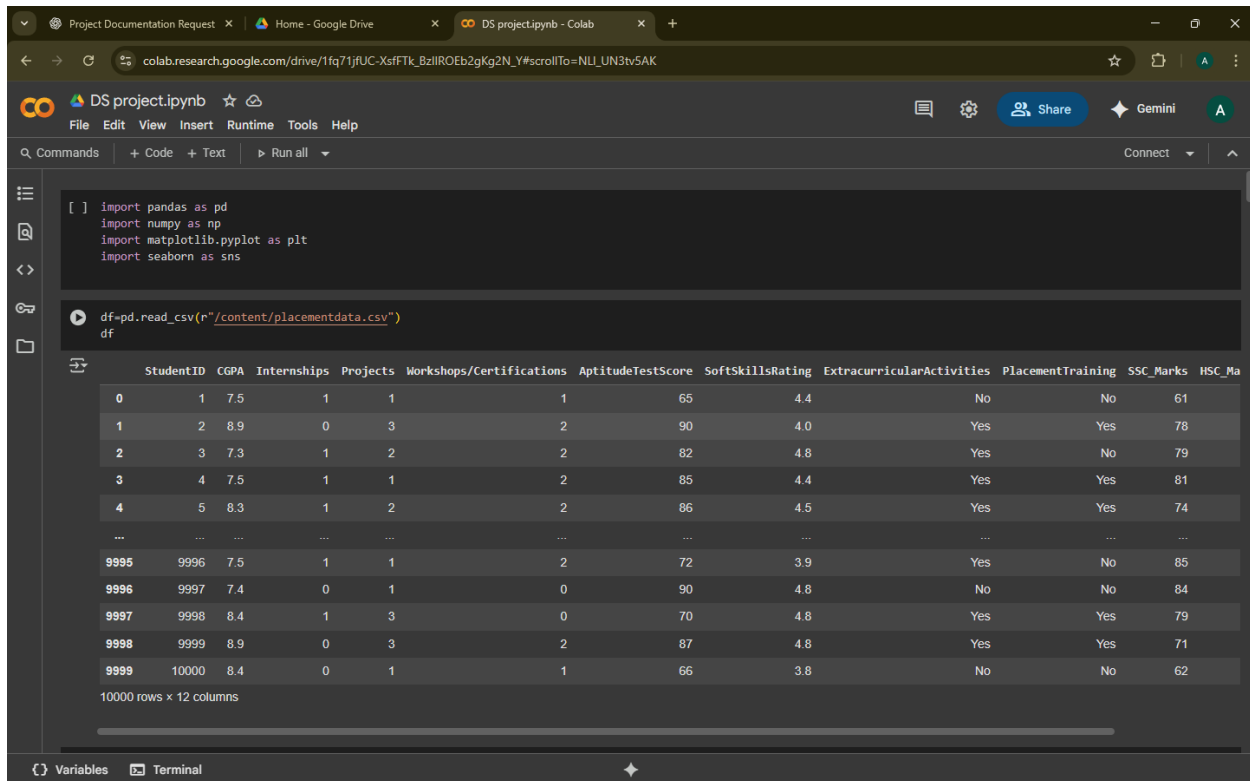
y_pred = model.predict(x_test)

from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)

*print(f"Accuracy: {accuracy * 100:.2f}%")*

A.2 SCREENSHOTS Include graphs of data distributions, heatmaps, confusion matrices, and prediction outputs from the notebook.



The screenshot shows a Google Colab notebook interface. The top bar includes the Colab logo, the notebook name "DS project.ipynb", and a "Share" button. The left sidebar contains icons for file explorer, code editor, and output. The main area displays the following code:

```
[ ] import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df=pd.read_csv(r"/content/placementdata.csv")
df
```

Below the code, a preview of the data is shown as a table with 12 columns: StudentID, CGPA, Internships, Projects, Workshops/Certifications, AptitudeTestScore, SoftSkillsRating, ExtracurricularActivities, PlacementTraining, SSC_Marks, and HSC_Ma. The table displays rows 0 through 9999, with a summary indicating "10000 rows x 12 columns".

	StudentID	CGPA	Internships	Projects	Workshops/Certifications	AptitudeTestScore	SoftSkillsRating	ExtracurricularActivities	PlacementTraining	SSC_Marks	HSC_Ma
0	1	7.5	1	1	1	65	4.4	No	No	61	
1	2	8.9	0	3	2	90	4.0	Yes	Yes	78	
2	3	7.3	1	2	2	82	4.8	Yes	No	79	
3	4	7.5	1	1	2	85	4.4	Yes	Yes	81	
4	5	8.3	1	2	2	86	4.5	Yes	Yes	74	
...	...	...	...	...	...	...	...	...	...	...	...
9995	9996	7.5	1	1	2	72	3.9	Yes	No	85	
9996	9997	7.4	0	1	0	90	4.8	No	No	84	
9997	9998	8.4	1	3	0	70	4.8	Yes	Yes	79	
9998	9999	8.9	0	3	2	87	4.8	Yes	Yes	71	
9999	10000	8.4	0	1	1	66	3.8	No	No	62	

At the bottom of the interface, there are tabs for "Variables" and "Terminal".

```
df.columns
```

```
Index(['StudentID', 'CGPA', 'Internships', 'Projects',  
      'Workshops/Certifications', 'AptitudeTestScore', 'SoftSkillsRating',  
      'ExtracurricularActivities', 'PlacementTraining', 'SSC_Marks',  
      'HSC_Marks', 'PlacementStatus'],  
      dtype='object')
```

```
[ ] df.drop(columns=['StudentID'], inplace=True)
```

```
[ ] df.info()
```

```
<class 'pandas.core.frame.DataFrame'  
RangeIndex: 10000 entries, 0 to 9999  
Data columns (total 11 columns):  
#   Column                                Non-Null Count  Dtype  
---  --  
0   CGPA                                  10000 non-null  float64  
1   Internships                          10000 non-null  int64  
2   Projects                             10000 non-null  int64  
3   Workshops/Certifications              10000 non-null  int64  
4   AptitudeTestScore                    10000 non-null  int64  
5   SoftSkillsRating                     10000 non-null  float64  
6   ExtracurricularActivities            10000 non-null  object  
7   PlacementTraining                    10000 non-null  object  
8   SSC_Marks                            10000 non-null  int64  
9   HSC_Marks                            10000 non-null  int64  
10  PlacementStatus                      10000 non-null  object  
dtypes: float64(2), int64(6), object(3)  
memory usage: 859.5+ KB
```

```
[ ] df.isna().sum()
```

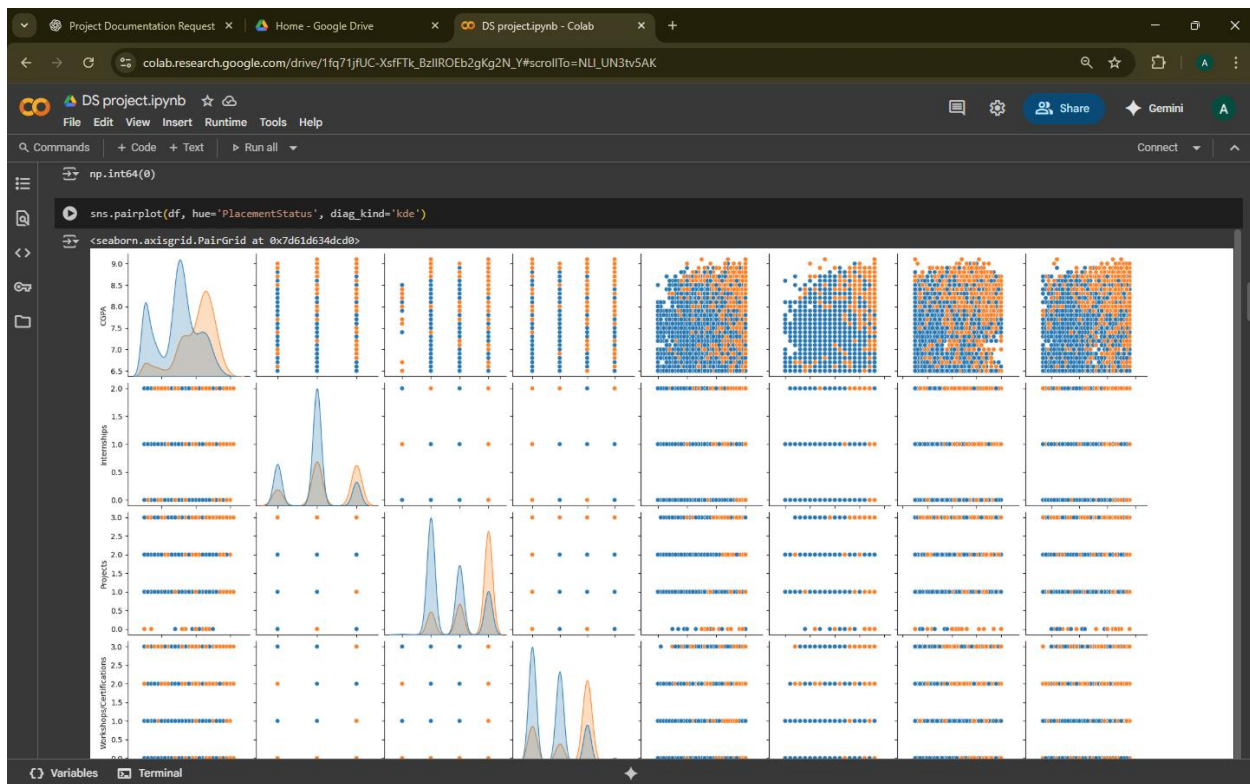
```
0
```

```
df.isna().sum()
```

```
0  
CGPA 0  
Internships 0  
Projects 0  
Workshops/Certifications 0  
AptitudeTestScore 0  
SoftSkillsRating 0  
ExtracurricularActivities 0  
PlacementTraining 0  
SSC_Marks 0  
HSC_Marks 0  
PlacementStatus 0  
dtype: int64
```

```
[ ] df.describe()
```

	CGPA	Internships	Projects	Workshops/Certifications	AptitudeTestScore	SoftSkillsRating	SSC_Marks	HSC_Marks
count	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	7.698010	1.049200	2.026600	1.013200	79.449900	4.323960	69.159400	74.501500
std	0.640131	0.665901	0.867968	0.904272	8.159997	0.411622	10.430459	8.919527



```

from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
le.fit(df['ExtracurricularActivities'])
df['ExtracurricularActivities'] = le.transform(df['ExtracurricularActivities'])

/tp/ipython-input-13-1356828198.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df['ExtracurricularActivities'] = le.transform(df['ExtracurricularActivities'])

le.fit(df['PlacementTraining'])
df['PlacementTraining'] = le.transform(df['PlacementTraining'])
le.fit(df['PlacementStatus'])
df['PlacementStatus'] = le.transform(df['PlacementStatus'])
df

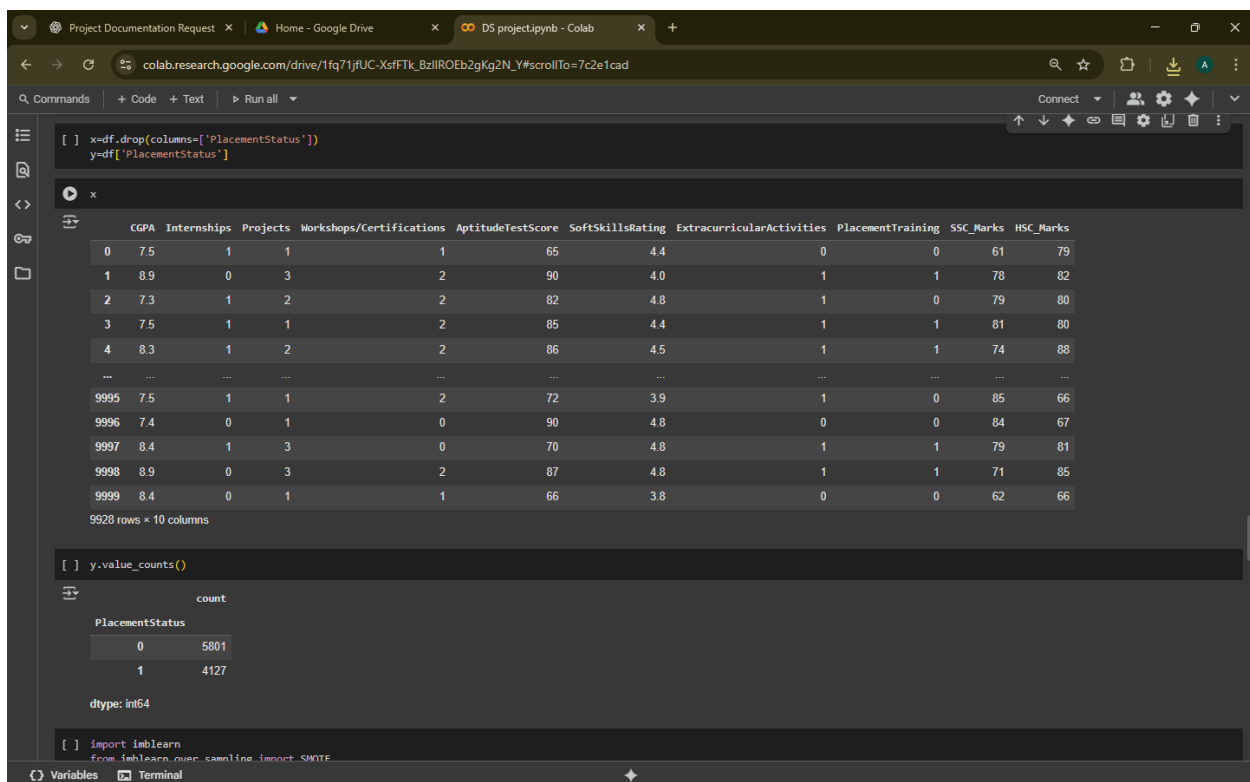
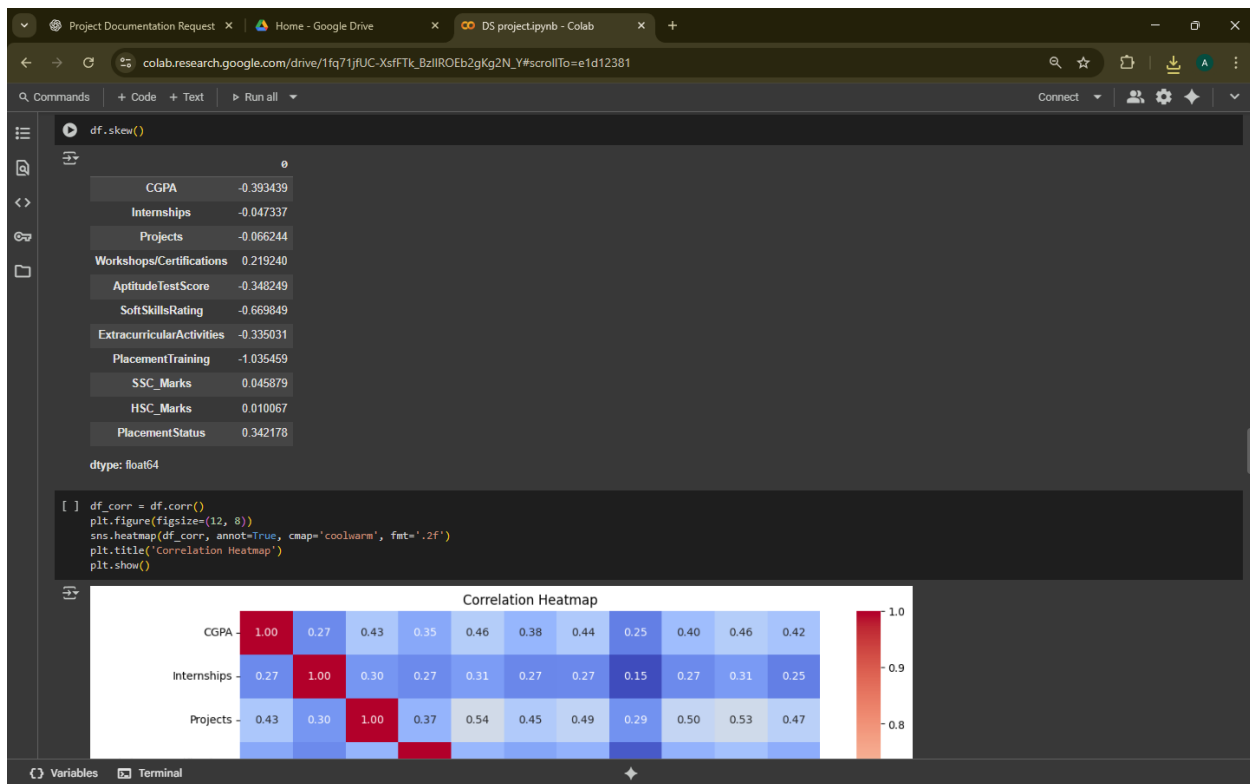
/tp/ipython-input-14-2481535929.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df['PlacementTraining'] = le.transform(df['PlacementTraining'])
/tp/ipython-input-14-2481535929.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df['PlacementStatus'] = le.transform(df['PlacementStatus'])

```

	CGPA	Internships	Projects	Workshops/Certifications	AptitudeTestScore	SoftSkillsRating	ExtracurricularActivities	PlacementTraining	SSC_Marks	HSC_Marks	PlacementStatus
0	7.5	1	1	1	65	4.4	0	0	61	79	0
1	8.9	0	3	2	90	4.0	1	1	78	82	1
2	7.3	1	2	2	82	4.8	1	0	79	80	0
3	7.5	1	1	2	85	4.4	1	1	81	80	1
4	8.3	1	2	2	86	4.5	1	1	74	88	1



```

Project Documentation Request X Home - Google Drive X DS projectjynb - Colab X +
colab.research.google.com/drive/1fq71jRUC-XsfFTk_BzIIROEb2gKg2N_Y#scrollTo=7c2e1cad

Commands + Code + Text Run all Connect
dtype: int64

[ ] import imblearn
from imblearn.over_sampling import SMOTE
smote = SMOTE()
x, y = smote.fit_resample(x, y)

y.value_counts()

count
PlacementStatus
0 5801
1 5801

dtype: int64

[ ] from sklearn.preprocessing import RobustScaler
sc = RobustScaler()
sc.fit(x)
x_scaled = sc.transform(x)
x_scaled

array([[ -0.33806085,  0.    , -0.5    , ..., -1.    ,
        -0.55   ,  0.23529412,  0.    , ...,  0.    ,
        [ 1.23955644, -1.    ,  0.5    , ...,  0.    ,
        0.3    ,  0.41176471],
        [-0.56343474,  0.    ,  0.    , ..., -1.    ,
        0.35   ,  0.29411765],
        ...,
        [ 0.65357645,  1.    ,  0.5    , ...,  0.    ,
        0.2    ,  0.70588235],
        [ 0.73404212,  1.    ,  0.5    , ...,  0.    ,
        -0.65   ,  0.52941176],
        [ 0.7044402 , -1.    ,  0.5    , ...,  0.    ,
        0.35   ,  0.41176471]])

[ ] from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x_scaled, y, test_size=0.2, random_state=16)
from sklearn.linear_model import LogisticRegression

```

```

Project Documentation Request X Home - Google Drive X DS projectjynb - Colab X +
colab.research.google.com/drive/1fq71jRUC-XsfFTk_BzIIROEb2gKg2N_Y#scrollTo=7c2e1cad

Commands + Code + Text Run all Connect
x_train, x_test, y_train, y_test = train_test_split(x_scaled, y, test_size=0.2, random_state=10)
[ ] from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy * 100:.2f}%--{i}")

Accuracy: 80.57%--PlacementStatus

[ ] from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x_scaled, y, test_size=0.2, random_state=16)
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(penalty='elasticnet', l1_ratio=1, solver='saga', random_state=100)
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy * 100:.2f}%--{i}")

Accuracy: 80.61%--PlacementStatus

[ ] x_train, x_test, y_train, y_test = train_test_split(x_scaled, y, test_size=0.2, random_state=4)
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(random_state=93)
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy * 100:.2f}%--{i}")

Accuracy: 81.00%--PlacementStatus

[ ] x_train, x_test, y_train, y_test = train_test_split(x_scaled, y, test_size=0.2, random_state=42)
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(criterion='gini', max_features='sqrt', random_state=0)
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred)

```

```

x_train, x_test, y_train, y_test = train_test_split(x_scaled, y, test_size=0.2, random_state=42)
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(criterion='gini', max_features='sqrt', random_state=0)
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy * 100:.2f}%")

Accuracy: 80.96%

[ ] pip install xgboost

Requirement already satisfied: xgboost in /usr/local/lib/python3.11/dist-packages (2.1.4)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from xgboost) (2.0.2)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.11/dist-packages (from xgboost) (2.21.5)
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages (from xgboost) (1.15.3)

[ ] x_train, x_test, y_train, y_test = train_test_split(x_scaled, y, test_size=0.2, random_state=42)
from xgboost import XGBClassifier
model = XGBClassifier()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy * 100:.2f}%")

Accuracy: 81.00%

[ ] from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x_scaled, y, test_size=0.2, random_state=16)
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(penalty='elasticnet', l1_ratio=1, solver='saga', random_state=100)
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy * 100:.2f}%--(i)")
import pandas as pd
cGPA = float(input("Enter CGPA: "))
Internships = int(input("Enter number of internships: "))

```

```

from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x_scaled, y, test_size=0.2, random_state=16)
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(penalty='elasticnet', l1_ratio=1, solver='saga', random_state=100)
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy * 100:.2f}%--(i)")
import pandas as pd
cGPA = float(input("Enter CGPA: "))
Internships = int(input("Enter number of internships: "))
Projects = int(input("Enter number of projects : "))
Workshops = int(input("Enter number of workshops/certifications: "))
AptitudeTestScore = int(input("Enter Aptitude Test Score: "))
SoftSkillsRating = float(input("Enter Soft Skills Rating: "))
ExtracurricularActivities = int(input("Enter of extracurricular activities(1:yes, 0:no): "))
PlacementTraining = int(input("Enter Placement Training Score(1:yes, 0:no): "))
sSC_Marks = int(input("Enter SSC Marks: "))
hSC_Marks = int(input("Enter HSC Marks: "))
y=("CGPA":cGPA, "Internships": Internships, "Projects": Projects,
  "Workshops/Certifications":Workshops, "AptitudeTestScore": AptitudeTestScore,
  "SoftSkillsRating": SoftSkillsRating, "ExtracurricularActivities": ExtracurricularActivities,
  "PlacementTraining": PlacementTraining, "SSC_Marks": sSC_Marks, "HSC_Marks": hSC_Marks)
df=pd.DataFrame([y])
print(df)
y_pred = model.predict(sc.transform(df))
y_pred
if y_pred[0] == 1:
    print("Candidate is likely to be placed.")
else:
    print("Candidate is unlikely to be placed.")

Accuracy: 80.61%--PlacementStatus
Enter CGPA: 8.3
Enter number of internships: 2
Enter number of projects : 2
Enter number of workshops/certifications: 10
Enter Aptitude Test Score: 85
Enter Soft Skills Rating: 90
Enter of extracurricular activities(1:yes, 0:no): 1

```

The screenshot displays a Google Colab notebook interface. The top navigation bar shows the URL `colab.research.google.com/drive/1fq71jFUC-XsfFTk_BzIIROEb2gKg2N_Y#scrollTo=7c2e1cad`. The notebook contains a code cell with the following Python code:

```
"SoftSkillsRating": SoftSkillsRating, "ExtracurricularActivities": ExtracurricularActivities,
"PlacementTraining": PlacementTraining, "SSC_Marks": sSC_Marks, "HSC_Marks": hSC_Marks}

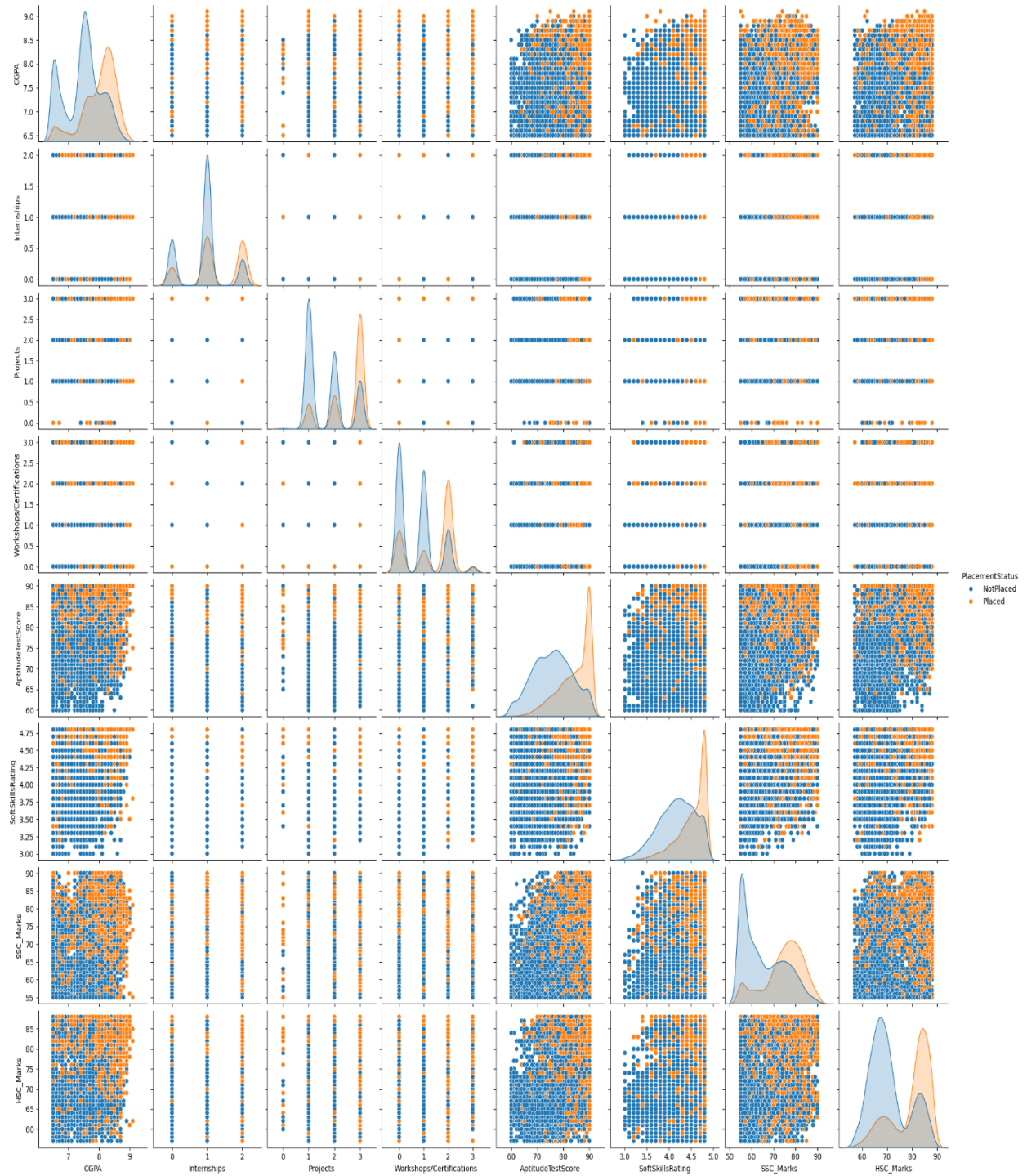
df=pd.DataFrame([y])
print(df)
y_pred = model.predict(sc.transform(df))
y_pred
if y_pred[0] == 1:
    print("Candidate is likely to be placed.")
else:
    print("Candidate is unlikely to be placed.")
```

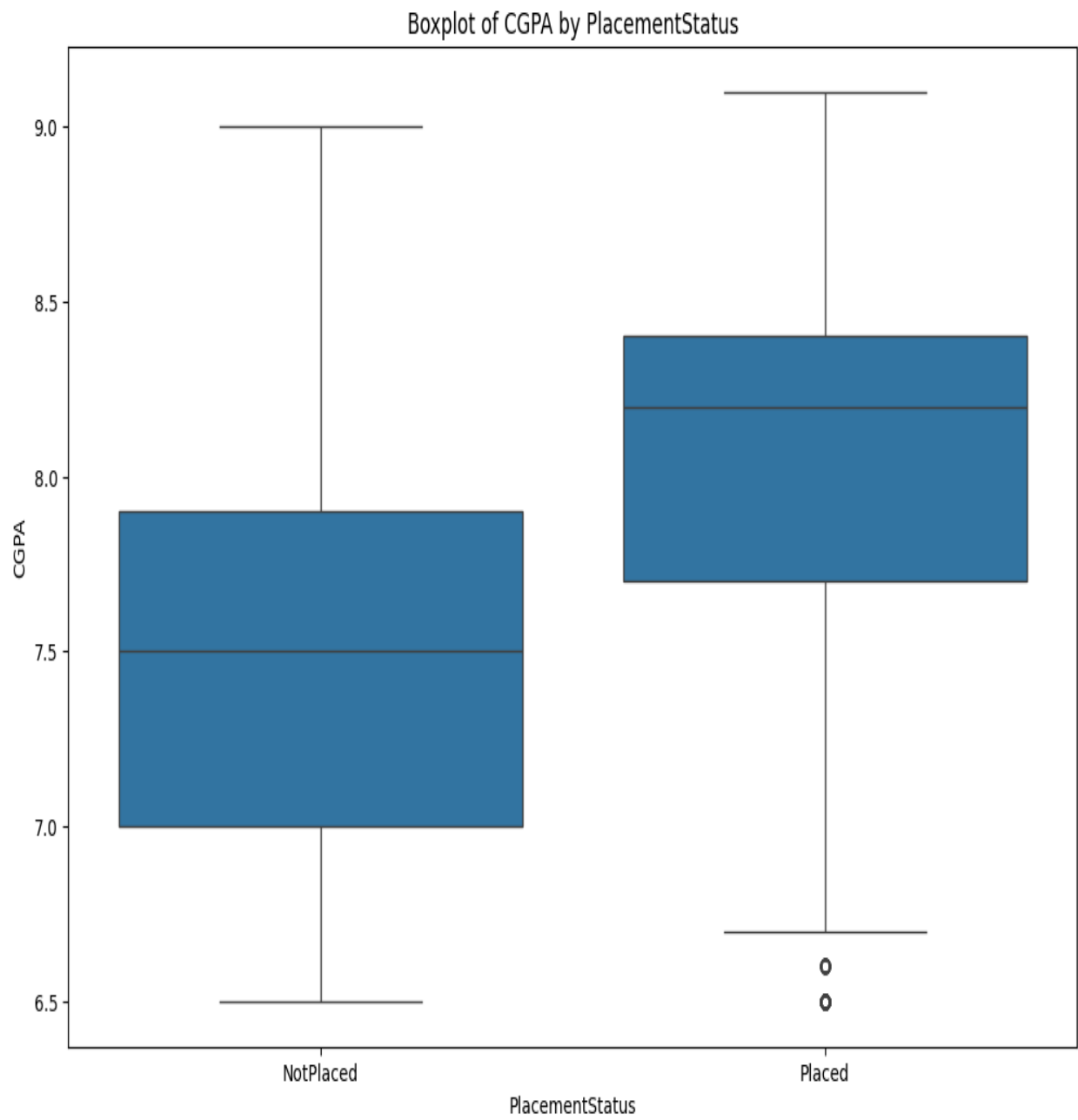
Below the code cell, the output shows the model's accuracy and a series of user inputs for a prediction:

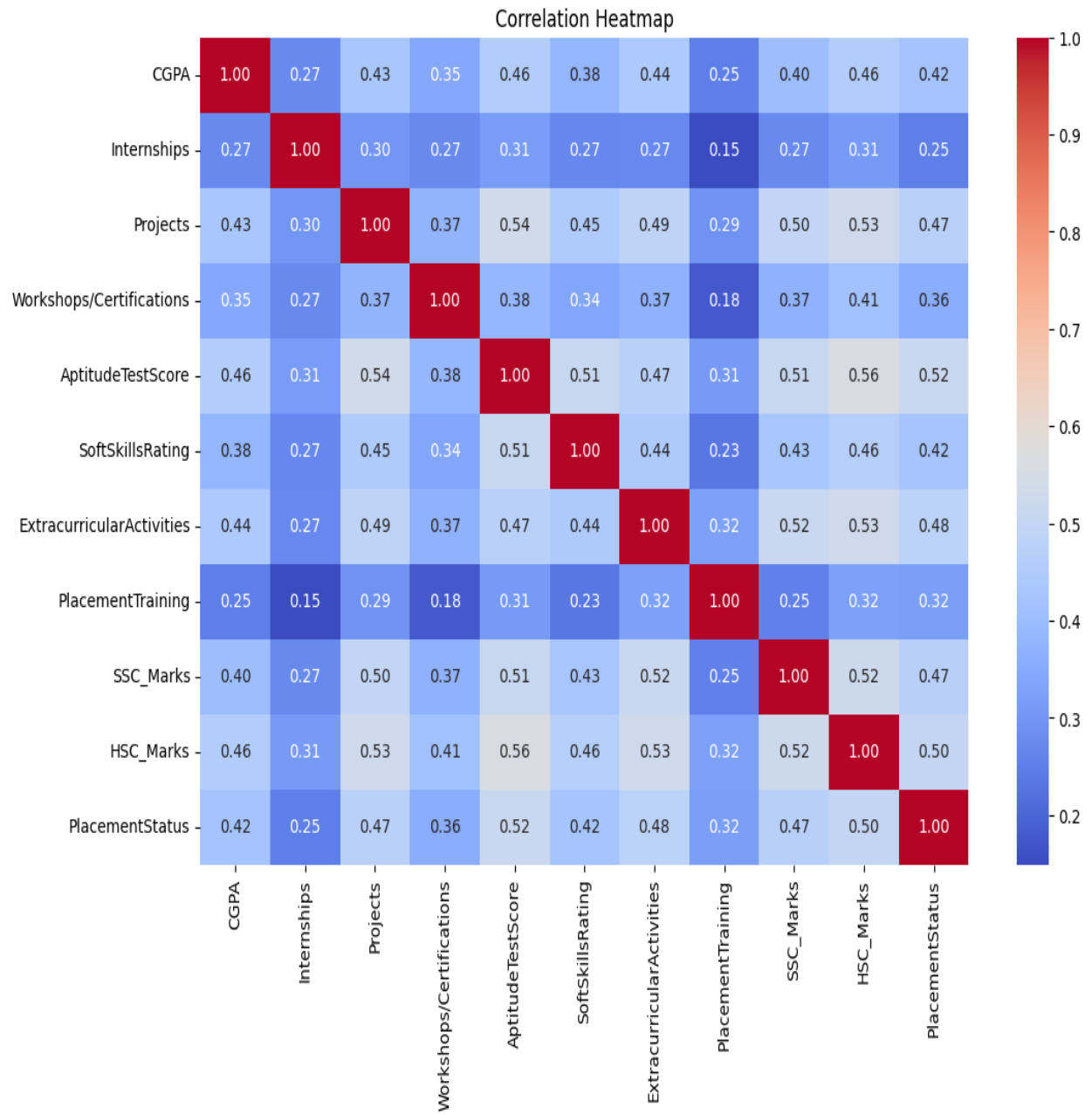
```
Accuracy: 80.61%-PlacementStatus
Enter CGPA: 8.3
Enter number of internships: 2
Enter number of projects : 2
Enter number of workshops/certifications: 10
Enter Aptitude Test Score: 85
Enter Soft Skills Rating: 90
Enter of extracurricular activities(1:yes , 0:no): 1
Enter Placement Training Score(1:yes , 0:no): 1
Enter SSC Marks: 65
Enter HSC Marks: 65
CGPA Internships Projects Workshops/Certifications AptitudeTestScore \
0 8.3 2 2 10 85
SoftSkillsRating ExtracurricularActivities PlacementTraining SSC_Marks \
0 90.0 1 1 65
HSC_Marks
0 65
Candidate is likely to be placed.
```

A Snipping Tool window is overlaid on the bottom right, displaying the message: "Screenshot copied to clipboard. Automatically saved to screenshots folder. Mark-up and share".

PLOTTING







REFERENCES

1. Çakıt, E. and Dağdeviren, M., 2022. Predicting the percentage of student placement: A comparative study of machine learning algorithms. *Education and Information Technologies*, 27(1), pp.997-1022.
2. Bhoite, Sachin, et al. "Predictive analytics model of an engineering and technology campus placement." *Proceeding of International Conference on Computational Science and Applications: ICCSA 2021*. Singapore: Springer Nature Singapore, 2022.
3. Nagaria, Jumana. "Utilizing exploratory data analysis for the prediction of campus placement for educational institutions." *2020 11th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*. IEEE, 2020.
4. Palle, Ramya Mahitha, et al. "Campus Placement Analysis: Predictive Insights with Machine Learning Models." *2025 3rd International Conference on Advancements in Electrical, Electronics, Communication, Computing and Automation (ICAECA)*. IEEE, 2025.
5. Jain, Prayush, Sandali Khare, and Mahendra Kumar Gourisaria. "A data mining solution to predict campus placement." *2021 IEEE 4th International Conference on Computing, Power and Communication Technologies (GUCON)*. IEEE, 2021.
6. Kumar, Nikhil, Ajay Shanker Singh, and E. Rajesh. "Campus placement predictive analysis using machine learning." *2020 2nd international conference on advances in computing, communication control and networking (ICACCCN)*. IEEE, 2020.
7. Surya, M. Siva, M. Sathish Kumar, and D. Gandhimathi. "Student placement prediction using supervised machine learning." *2022 2nd International Conference on Advance Computing and Innovative Technologies in Engineering (ICACITE)*. IEEE, 2022.
8. Muraina IO, Agoi MA, Omorojor BO. Forecasting students' job placement using data science paradigms. *International Journal of Education and Learning Systems*. 2022;7.

12.07.2025,

Coimbatore.

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Mr. ABDUL RAHEEM K** (Reg No:730323243001), of **III B.Tech Artificial Intelligence and Data Science** student from **Kangayam Institute of Technology** has successfully completed his Internship Training and Mini Project Entitled in **"ON CAMPUS STUDENTS PLAMENT PREDICTION USING PYTHON WITH DATA SCIENCE"** at our organization from **26.06.2025 to 12.07.2025**.

During this period, he was sincere and regular in attending all phases of Intern program.

For Accent Techno Soft (ATS),



[Signature]
12/07

Authorized Signatory

#203, Nehru Street, Ram Nagar, Coimbatore - 641009

www.accenttechnosoft.com

0422 - 4212232

info@accenttechnosoft.com